



Time Series Anomaly Detection with Harbinger



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Roadmap

A structured progression of methods in the `harbinger` package (R) — from the simplest baseline to deep architectures and multivariate analysis.

Fundamentals

`hanr_histogram()`

Flags observations in low-density histogram bins — no temporal model required.

Functions: `hanr_histogram()`

Supervised

`hanr_ml()` / `hanc_ml()`

`hanr_ml()` uses regression prediction error; `hanc_ml()` uses labeled classification probability.

Functions: `ts_elm()`, `ts_mlp()`, `ts_rf()`, `ts_svm()`, `ts_lstm()`, `ts_conv1d()` / `cla_dtree()`, `cla_knn()`, `cla_nb()`, `cla_rf()`, `cla_svm()`, `cla_mlp()`

Ensembles & Deep

`har_ensemble()` / `han_autoencoder()`

`har_ensemble()` combines detector votes; `han_autoencoder()` flags high reconstruction error on unseen windows.

Functions: `har_ensemble()`, `autoenc_ed`, `autoenc_conv_ed`, `autoenc_lstm_ed`, `autoenc_variational_ed`, `autoenc_adv_ed`

Residual Models

`hanr_*`

Fits a model of expected behavior and flags large deviations (residuals) as anomalies.

Functions: `hanr_arima()`, `hanr_fbiad()`, `hanr_emd()`, `hanr_fft*`, `hanr_garch()`, `hanr_remd()`, `hanr_rtad()`, `hanr_wavelet()`

Clustering

`hanct_*`

Assigns observations or subsequences to clusters; large distance to nearest centroid signals an anomaly or discord.

Functions: `hanct_kmeans()`, `hanct_dtw()`

Multivariate

`hmu_*`

Detects anomalies in the joint behavior of multiple simultaneous variables via PCA reconstruction error.

Functions: `hmu_pca()`

📄 Run the full examples and explore the source code at: <https://github.com/cefet-rj-dal/harbinger/tree/master/examples>

Standard Harbinger Workflow

Every detector in Harbinger follows this 5-step workflow. The consistency of the interface makes it easy to compare methods from completely different families with minimal code changes — only the model constructor needs to change.

Carregar

Visualizar

Configurar

Detectar e Avaliar

Setup and detection code

```
library(daltoolbox)
library(harbinger)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Visualize raw series
har_plot(harbinger(), dataset$serie)

# --- method-specific (changes per slide) ---
# model <- hanr_<method>(...)
# model <- fit(model, dataset$serie)
# -----

# Detect (common to all methods)
detection <- detect(model, dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
```

Evaluate and plot

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

# Plot detections on the series
har_plot(model, dataset$serie,
  detection, dataset$event)

# Plot residuals with threshold line
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

How to read this block

This block is **identical across all detectors** — only the model constructor (shown in each method slide) needs to change.

- `detect()` returns a dataframe with an `event` column
- `filter(event == TRUE)` shows flagged indices
- `evaluate()` computes `confMatrix` (TP / FP / FN / TN)
- First `har_plot()` overlays detections on the original series
- Second `har_plot()` shows residual magnitudes with the decision threshold line (not available for all methods)

01 · hanr_histogram() – Histogram Baseline

The `hanr_histogram()` detector builds a frequency histogram from the observed values of a time series and flags any points that fall in low-density bins (below a configurable threshold, default 0.05) or outside the observed range as anomalies. It requires no model fitting and serves as a purely data-driven baseline with no assumptions about temporal structure.

 **Key concept:** `hanr_histogram()` is a distribution-based baseline — it makes no assumptions about temporal structure and focuses solely on the empirical value distribution, making it a transparent and interpretable starting point for anomaly detection.

```
model <- hanr_histogram(  
  density_threshold = 0.05)  
model <- fit(model,  
  dataset$serie)
```

```
## Detections:  
## idx event  type  
##   1  TRUE anomaly  
##  50  TRUE anomaly  
  
## confMatrix:  
## detection TRUE FALSE  
##    TRUE  1   1  
##   FALSE  0  99
```

Step-by-step

- `density_threshold = 0.05` marks bins with frequency below 5% as anomalous
- `fit()` is a no-op here, but preserves the standard workflow pattern
- `detect()` flags points in low-density bins or outside the observed range
- Anomaly score = bin density (lower density → more anomalous)

02 · hanr_arima() – ARIMA Residuals

ARIMA (AutoRegressive Integrated Moving Average) models a time series as a linear combination of its own past values (AR), past forecast errors (MA), and differencing to achieve stationarity (I). The model captures linear autocorrelation structure in the series. Large deviations from the model's expected values — standardized residuals — indicate anomalies. Best for stationary or easily differenced series with linear temporal dependence. Ref: Box & Jenkins (1976).

❏ **Ref:** Box & Jenkins (1976). Thresholds via `harutils()`.

```
model <- hanr_arima()
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##   TRUE   1    0
##  FALSE   0  100
```

Implementation

- `hanr_arima()` requires no arguments — order (p,d,q) is selected automatically via `auto.arima()` from the `forecast` package
- `fit()` estimates the ARIMA parameters and stores the fitted model
- `detect()` computes standardized residuals (observed – predicted) / σ for each observation
- `harutils()` applies the threshold to flag anomalies
- Anomaly score = |standardized residual|
- The second `har_plot()` call displays residual magnitudes with the decision threshold line

03 · hanr_fbiad() – Forward-Backward Inertia Anomaly Detection

FBIAD (Forward-Backward Inertia Anomaly Detector) detects anomalies by comparing each observation against the local trend in both temporal directions. For each point, it computes the mean of a **forward window** (future values) and a **backward window** (past values). The inertia score measures how much the point deviates from both local trends simultaneously. A point is flagged only if it breaks **both** forward and backward inertia — this dual requirement reduces false positives compared to single-direction methods. Best suited for series with smooth local trends where abrupt point anomalies stand out.

📄 Ref: Lima et al., IJCNN 2022.

```
model <- hanr_fbiad()
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##    TRUE   1    0
##   FALSE   0  100
```

Implementation

- `hanr_fbiad()` requires no arguments — window sizes are determined automatically
- `fit()` stores the series statistics needed for inertia computation
- `detect()` computes forward and backward window means for each point and derives the inertia score
- A point is flagged only when it exceeds the threshold in both directions
- `harutils()` applies the threshold to the aggregated scores
- Anomaly score = combined deviation from forward and backward local means

04 · hanr_emd() – Empirical Mode Decomposition

EMD (Empirical Mode Decomposition) is a data-driven decomposition method that breaks a signal into **Intrinsic Mode Functions (IMFs)** — oscillatory components extracted directly from the data without requiring a predefined basis (unlike Fourier or wavelets). Each IMF captures oscillations at a specific time scale. High-frequency IMFs reflect rapid local changes; anomalies manifest as elevated energy in these fine-scale components.

CEEMD (Complementary Ensemble EMD) is a noise-assisted variant that improves mode mixing. Best for non-stationary and non-linear series where ARIMA assumptions do not hold.

❏ **Ref:** Huang et al. (1998). CEEMD improves on standard EMD by reducing mode mixing through noise-assisted decomposition.

```
model <- hanr_emd()
model <- fit(model, dataset$serie)
```

```
## idx event  type
## 50 TRUE anomaly

## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100
```

Implementation

- `hanr_emd()` uses CEEMD via the `hht` package (`hht::CEEMD`) — no arguments required
- `fit()` decomposes the series into IMFs and stores the decomposition
- `detect()` isolates high-frequency IMF components and computes their energy
- `harutils()` applies the threshold to flag anomalies
- Anomaly score = energy of high-frequency IMF components
- Requires the `hht` package to be installed

05 · hanr_fft() – Spectral Filtering via FFT

FFT (Fast Fourier Transform) decomposes a time series into its frequency components. Normal behavior is typically captured by low-frequency components (slow trends, seasonality); anomalies manifest as high-frequency spikes. A high-pass filter removes the low-frequency components, leaving a residual that captures abrupt deviations.

The key design choice across FFT variants is how the frequency cutoff is selected — this gives rise to six variants: `hanr_fft()` (fixed power-spectrum cutoff), `hanr_fft_amoc()`, `hanr_fft_binseg()`, `hanr_fft_amoc_cusum()`, `hanr_fft_binseg_cusum()`, `hanr_fft_sma()`. Best for series with periodic or quasi-periodic structure.

❏ **Ref:** Cooley & Tukey (1965). The six variants differ only in how the frequency cutoff is selected — see slides 06–10.

```
model <- hanr_fft()
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##    TRUE   1    0
##   FALSE   0  100
```

Implementation

- `hanr_fft()` requires no arguments
- `fit()` computes the power spectrum and selects the cutoff based on the dominant frequency
- `detect()` applies the high-pass filter, reconstructs the high-frequency signal via inverse FFT, and thresholds residuals via `harutils()`
- Anomaly score = magnitude of the high-frequency residual
- All six FFT variants share the same `fit()→detect()` interface — only the cutoff selection strategy differs

06 · hanr_fft_amoc() – FFT with AMOC Selection

AMOC (At Most One Change) is a changepoint detection method that assumes at most one structural break exists in the signal. Applied to the FFT power spectrum, it identifies the single frequency at which the spectral energy changes most abruptly — this frequency becomes the cutoff for the high-pass filter. AMOC is efficient and well-suited for series with a single dominant regime shift in the frequency domain. It is less appropriate when multiple spectral changes are present (use BinSeg variants instead).

📌 **Key concept:** AMOC is from the `changepoint` R package. Efficient $O(n)$ changepoint detection.

```
model <- hanr_fft_amoc()
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##    TRUE   1    0
##   FALSE   0  100
```

Implementation

- `hanr_fft_amoc()` requires no arguments
- `fit()` computes the FFT power spectrum and applies AMOC to find the single changepoint
- `detect()` uses the AMOC-selected cutoff to filter the spectrum, reconstructs the high-frequency residual, and thresholds via `harutils()`
- Anomaly score = magnitude of the high-frequency residual after AMOC-based filtering
- Assumes at most one spectral changepoint — use BinSeg variants for multiple changepoints

07 · hanr_fft_amoc_cusum() – FFT + CUSUM + AMOC

CUSUM (Cumulative Sum) is a sequential analysis technique that accumulates deviations from a target value over time. Applied to the FFT power spectrum before AMOC changepoint selection, CUSUM amplifies gradual, persistent shifts in spectral energy that would be missed by a single-point comparison. This makes the combined FFT+CUSUM+AMOC approach more sensitive to smooth regime changes in the frequency domain — where the spectral energy drifts gradually rather than shifting abruptly.

 **Key concept:** CUSUM pre-processing amplifies slow spectral drifts before AMOC — more robust than plain AMOC for gradual regime changes.

```
model <-  
hanr_fft_amoc_cusum()  
model <- fit(model,  
dataset$serie)
```

Output:

```
## idx event  type  
## 50  TRUE anomaly  
  
## detection TRUE FALSE  
##   TRUE  1    0  
##  FALSE  0  100
```

Implementation

- `hanr_fft_amoc_cusum()` requires no arguments
- `fit()` computes the FFT power spectrum, applies CUSUM accumulation, then AMOC changepoint detection
- Pipeline: FFT → CUSUM on spectrum → AMOC → inverse FFT → `harutils()` threshold
- `detect()` applies the full pipeline and returns flagged indices
- Anomaly score = magnitude of the high-frequency residual after CUSUM+AMOC filtering

08 · hanr_fft_binseg() – FFT with Binary Segmentation

Binary Segmentation (BinSeg) is a recursive changepoint detection method. It finds the single point of maximum contrast in the signal, splits the series at that point, and recurses on each segment — allowing it to detect multiple changepoints. Applied to the FFT power spectrum, BinSeg identifies the optimal frequency cutoff more flexibly than AMOC, which is limited to a single changepoint. BinSeg is preferred when the power spectrum contains multiple potential breakpoints.

📌 **Key concept:** BinSeg is from the `changepoint` R package. Use when the power spectrum has multiple potential breakpoints.

```
model <- hanr_fft_binseg()
model <- fit(model, dataset$serie)
```

Output:

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##    TRUE   1    0
##   FALSE   0  100
```

Implementation

- `hanr_fft_binseg()` requires no arguments
- `fit()` computes the FFT power spectrum and applies Binary Segmentation to find the optimal cutoff
- `detect()` filters the spectrum at the BinSeg-selected cutoff, reconstructs the high-frequency residual, and thresholds via `harutils()`
- Anomaly score = magnitude of the high-frequency residual
- More flexible than AMOC — handles multiple spectral changepoints

09 · hanr_fft_binseg_cusum() – FFT + CUSUM + BinSeg

`hanr_fft_binseg_cusum()` combines CUSUM accumulation with Binary Segmentation on the FFT power spectrum. CUSUM amplifies gradual, persistent spectral drifts before BinSeg searches for the optimal cutoff — making this the most robust FFT variant for series where spectral change is both gradual and multi-modal. It is the recommended choice when neither plain AMOC (single changepoint) nor plain BinSeg (abrupt changes) is sufficient.

📌 **Key concept:** Use this variant when the spectrum has both gradual drift and multiple potential changepoints.

```
model <- hanr_fft_binseg_cusum()
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##    TRUE   1    0
##   FALSE   0  100
```

Implementation

- `hanr_fft_binseg_cusum()` requires no arguments
- `fit()` applies the full pipeline: FFT → CUSUM on spectrum → BinSeg → cutoff selection
- `detect()` reconstructs the high-frequency residual and thresholds via `harutils()`
- Anomaly score = magnitude of the high-frequency residual after CUSUM+BinSeg filtering
- Most robust FFT variant — combines gradual drift detection (CUSUM) with flexible cutoff selection (BinSeg)

10 · `hanr_fft_sma()` – FFT + Adaptive Moving Average

This FFT variant uses the dominant frequency of the series — the spectral peak with the highest energy — to define the window size of an adaptive Simple Moving Average (SMA). The SMA smooths the series at the scale of its dominant oscillation; the residual (original – smoothed) captures deviations that are abrupt relative to the dominant pattern. Unlike the other FFT variants that use changepoint methods, `fft_sma` derives the cutoff directly from the spectral peak — making it fully automatic and parameter-free.

❏ **The warning is benign.** It occurs because the SMA window may not divide evenly into the series length.

```
model <- hanr_fft_sma()
model <- fit(model, dataset$serie)
```

```
## Warning: longer object length is
## not a multiple of shorter object
```

```
## idx event  type
## 50 TRUE anomaly
```

```
## detection TRUE FALSE
##    TRUE  1    0
##   FALSE  0  100
```

Implementation

- `hanr_fft_sma()` requires no arguments
- `fit()` identifies the dominant frequency from the power spectrum and computes the corresponding SMA window size
- `detect()` applies the SMA, computes residuals (original – smoothed), and thresholds via `harutils()`
- The "longer object length" warning during `detect()` is expected — it arises from a length mismatch in the SMA convolution and does not affect the result
- Anomaly score = $|\text{original} - \text{SMA}(\text{original})|$

11 · hanr_garch() – Conditional Heteroscedasticity

GARCH (Generalized AutoRegressive Conditional Heteroscedasticity) models time-varying volatility — the phenomenon where the variance of a series changes over time (volatility clustering). The ARCH component captures how past squared residuals drive current variance; the GARCH component adds persistence by including past variance values.

Anomalies are observations whose deviation from the mean is large relative to the locally expected volatility — not relative to a global threshold. Best suited for financial or sensor data where variance changes over time.

📄 **Refs:** Engle (1982), Bollerslev (1986). Requires the `rugarch` package.

```
# sGARCH(1,1) with ARMA mean
model <- hanr_garch()
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##    TRUE   1    0
##   FALSE   0  100
```

Implementation

- `hanr_garch()` uses `sGARCH(1,1)` with an ARMA mean equation by default — no arguments required
- `fit()` estimates both the mean and variance equations simultaneously via the `rugarch` package
- `detect()` computes standardized residuals: $(\text{observed} - \text{mean}) / \text{conditional_std}$
- `harutils()` applies the threshold to the standardized residuals
- Anomaly score = $|\text{standardized residual}|$ relative to the conditional variance

12 · hanr_remd() – Robust EMD with ARIMA

REMD (Robust EMD) combines two complementary approaches in a two-stage pipeline. **Stage 1:** EMD decomposes the series into IMFs, capturing non-stationary oscillatory structure that linear models cannot represent. **Stage 2:** ARIMA models the autocorrelated dynamics of the residual component left after EMD — the part that EMD alone cannot explain. Anomalies are flagged where the combined residual (EMD + ARIMA) exceeds the threshold. The combination is more robust than either method alone, at the cost of higher sensitivity (may produce false positives on simple datasets). Best for non-stationary series with both oscillatory modes and autocorrelated residuals.

📄 **Ref:** Souza et al., IJCNN 2024. The 1 FP on this dataset reflects higher sensitivity — not a failure.

```
model <- hanr_remd()
model <- fit(model, dataset$serie)
```

Detections

```
## idx event  type
##  50  TRUE anomaly
##  54  TRUE anomaly

## detection TRUE FALSE
##    TRUE  1   1
##   FALSE  0  99
```

Implementation

- `hanr_remd()` requires no arguments — both stages are automatic
- `fit()` runs CEEMD decomposition (via `hht`) then fits ARIMA on the residual component
- `detect()` computes the combined residual and thresholds via `harutils()`
- Anomaly score = combined EMD + ARIMA residual magnitude
- Higher sensitivity than either method alone — threshold tuning may be needed
- Requires both `hht` and `forecast` packages

13 · hanr_rtad() – Adaptive Window-Based Detector

`hanr_rtad()` is an adaptive detector that scores each observation relative to its local context rather than a global model. It uses EMD decomposition to extract local structural components, then estimates robust dispersion (MAD — Median Absolute Deviation) within sliding windows. Points that deviate significantly from the locally expected behavior are flagged. This local adaptivity makes RTAD more flexible than FBIAD (which uses fixed window means) for series with irregular or non-stationary local structure. Best suited for series with mean or variance shifts over time.

📌 **Key concept:** MAD (Median Absolute Deviation) is more robust to outliers than standard deviation — making RTAD resistant to contamination in the training window.

```
model <- hanr_rtad()
model <- fit(model, dataset$serie)
```

Detections

```
## idx event  type
## 50  TRUE anomaly

## detection TRUE FALSE
##   TRUE  1    0
##  FALSE  0  100
```

Implementation

- `hanr_rtad()` requires no arguments — window size and dispersion estimation are automatic
- `fit()` runs EMD decomposition and stores the local structural components
- `detect()` estimates MAD within sliding windows and scores each point relative to local behavior
- `harutils()` applies the threshold to the local deviation scores
- Anomaly score = deviation from local adaptive model, normalized by robust dispersion (MAD)
- Requires the `hht` package for EMD

14 · hanr_wavelet() – Multiresolution Analysis

Wavelet decomposition breaks a signal into components at multiple time scales simultaneously — unlike FFT which only provides global frequency information. The Maximal Overlap Discrete Wavelet Transform (MODWT) produces detail coefficients at each resolution level: fine levels capture rapid local changes; coarse levels capture slow trends. Anomalies manifest as elevated localized energy in the high-frequency detail coefficients. A key advantage over FFT: wavelets preserve temporal localization — they reveal both *when* and *where* in the series the anomaly occurs. Best for transient or localized anomalies (spikes, bursts) in non-stationary series.

📄 **Ref:** Mallat, IEEE TPAMI 1989. **MODWT vs DWT:** MODWT is shift-invariant and works on any sample size — preferred for anomaly detection.

```
# Uses MODWT internally
model <- hanr_wavelet()
model <- fit(model, dataset$serie)
```

Output

```
## idx event  type
##  50  TRUE anomaly

## detection TRUE FALSE
##    TRUE  1    0
##   FALSE  0  100
```

Implementation

- `hanr_wavelet()` uses MODWT internally — no arguments required
- `fit()` decomposes the series into multiresolution detail bands using MODWT
- `detect()` aggregates high-frequency detail coefficients into a magnitude signal and thresholds via `harutils()`
- Anomaly score = localized energy in high-frequency wavelet detail coefficients
- Requires the `wavelets` package

15 · hanr_ml + ELM – Extreme Learning Machine

`hanr_ml()` is a wrapper that enables any DALToolbox regression model to be used for anomaly detection. The principle: train a regressor to predict the next value from a sliding window of past observations; at inference, compute the prediction residual at each step; observations whose residuals exceed a learned threshold are flagged as anomalies.

ELM (Extreme Learning Machine) is a single-hidden-layer feedforward network where input weights are randomly assigned and never updated — only the output weights are learned via least squares. This makes training extremely fast (no iterative optimization) but may sacrifice accuracy compared to fully trained networks.

❏ **Key concept:** ELM trades accuracy for speed. The random hidden layer avoids vanishing gradients and makes training $O(n)$.

```
# ELM como regressor
model <- hanr_ml(
  ts_elm(ts_norm_gminmax(),
    input_size = 4,
    nhid = 3,
    actfun = "purelin"))
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  52 TRUE anomaly

## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  1  99
```

Implementation

- Requires the `tspredict` package
- `ts_norm_gminmax()` embeds global min-max normalization in the pipeline
- `input_size=4` defines the sliding window length
- `nhid=3` sets the number of hidden neurons (fixed random weights)
- `actfun="purelin"` uses a linear activation function
- `fit()` trains only the output weights via least squares
- `detect()` computes residuals and thresholds via `harutils()`
- Detected `idx=52` instead of 50 — a 2-step offset (1 FN, 1 FP)

16 · hanr_ml + MLP – Multilayer Perceptron

MLP (Multilayer Perceptron) is a fully connected feedforward neural network with one or more hidden layers. Unlike ELM, all weights are learned through backpropagation and gradient descent — the network iteratively adjusts its parameters to minimize prediction error. This allows MLP to learn complex non-linear mappings between inputs and outputs. Used within `hanr_ml()`, MLP is trained to predict the next value from a sliding window; large prediction residuals signal anomalies. More accurate than ELM on structured series, but slower to train.

❏ **Key concept:** `decay > 0` adds L2 regularization — useful when the training series is short or noisy.

```
# MLP como regressor
model <- hanr_ml(
  ts_mlp(ts_norm_gminmax(),
    input_size = 3,
    size = 3,
    decay = 0))
model <- fit(model, dataset$serie)
```

```
## idx event  type
## 50  TRUE anomaly

## detection TRUE FALSE
##   TRUE  1    0
##  FALSE  0  100
```

Implementation

- Requires the `tspredict` package
- `ts_norm_gminmax()` embeds global min-max normalization
- `input_size=3` defines the sliding window
- `size=3` sets the number of hidden neurons
- `decay=0` disables L2 regularization (increase to prevent overfitting on larger datasets)
- `fit()` trains all weights via backpropagation
- `detect()` computes residuals and thresholds via `harutils()`
- Anomaly score = $|\text{predicted} - \text{observed}|$

17 · hanr_ml + Random Forest – Forest Regression

Random Forest is an ensemble method that builds multiple decision trees, each trained on a bootstrap sample of the data and a random subset of features. Predictions are averaged across all trees (for regression). The randomness in both data sampling and feature selection reduces variance and improves generalization compared to a single tree. Used within `hanr_ml()`, Random Forest is trained to predict the next value from a sliding window; large prediction residuals signal anomalies. Robust to noise in training data, but can be sensitive to local variations in the series — producing false positives when local patterns are irregular.

❏ **Key concept:** Increasing `ntree` improves stability; increasing `nodesize` reduces overfitting. On this dataset RF produced 2 FPs — tuning these parameters may help.

```
# Random Forest como regressor
model <- hanr_ml(
  ts_rf(ts_norm_gminmax(),
    input_size = 4,
    nodesize = 1,
    ntree = 20))
model <- fit(model, dataset$serie)
```

```
## idx event  type
## 75 TRUE anomaly
## 84 TRUE anomaly

## detection TRUE FALSE
##   TRUE  0   2
##   FALSE 1  98
```

Implementation

- requires the `tspredict` and `randomForest` packages
- `ts_norm_gminmax()` embeds normalization
- `input_size=4` defines the sliding window
- `ntree=20` sets the number of trees
- `nodesize=1` allows trees to grow deep (minimum terminal node size)
- `fit()` trains the forest on the windowed series
- `detect()` computes residuals and thresholds via `harutils()`
- anomaly score = $|\text{predicted} - \text{observed}|$

18 · hanr_ml + SVM – Support Vector Machine

Support Vector Regression (SVR) finds a regression function that deviates from the target values by at most ϵ (epsilon-insensitive loss). The kernel trick maps inputs into a high-dimensional feature space, enabling SVR to model non-linear relationships without explicitly computing the transformation. The RBF (Radial Basis Function) kernel is particularly effective for capturing non-linear temporal patterns. Used within `hanr_ml()`, SVR is trained to predict the next value from a sliding window; large prediction residuals signal anomalies. Normalization is required for SVM-based methods.

❏ **Key concept:** The RBF kernel parameter γ is set automatically by `e1071`. Normalization is mandatory for SVM — without it, the kernel distances are meaningless.

```
model <- hanr_ml(  
  ts_svm(ts_norm_gminmax(),  
    input_size = 4,  
    kernel = "radial")  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
## 50 TRUE anomaly  
  
## detection TRUE FALSE  
##   TRUE  1    0  
##  FALSE  0   100
```

Implementation

- Requires the `tspredict` and `e1071` packages
- `ts_norm_gminmax()` is required — SVM is sensitive to feature scale
- `input_size=4` defines the sliding window
- `kernel="radial"` selects the RBF kernel
- `cost (C)` can be tuned — higher C = tighter fit to training data (default used here)
- `fit()` trains the SVR model
- `detect()` computes residuals and thresholds via `harutils()`
- Anomaly score = $|\text{predicted} - \text{observed}|$

19 · hanr_ml + LSTM – Long Short-Term Memory

LSTM (Long Short-Term Memory) is a recurrent neural network architecture designed to capture long-range temporal dependencies. It introduces a memory cell and three gating mechanisms — **input gate** (what new information to store), **forget gate** (what to discard from memory), and **output gate** (what to output) — that control the flow of information across time steps. This solves the vanishing gradient problem of standard RNNs, enabling the network to learn dependencies spanning many time steps. Used within `hanr_ml()`, LSTM is trained to predict the next value from a sequence; large prediction residuals signal anomalies.

❏ **Key concept:** Reserve LSTM for series with genuine long-range dependencies. For locally-structured anomalies, Conv1D achieves better results with the same training budget.

```
# Requer: library(daltoolboxdp)
model <- hanr_ml(
  ts_lstm(ts_norm_gminmax(),
    input_size = 4,
    epochs = 10000))
model <- fit(model, dataset$serie)
```

```
## idx event  type
## 50 TRUE anomaly
## 54 TRUE anomaly

## detection TRUE FALSE
##   TRUE  1   1
##  FALSE  0  99
```

Implementation

- Requires `daltoolboxdp` and `tspredict` packages
- `ts_norm_gminmax()` embeds normalization
- `input_size = 4` defines the sequence window
- `epochs = 10000` — intensive training required for convergence
- `fit()` trains the LSTM via backpropagation through time (BPTT)
- `detect()` computes residuals and thresholds via `harutils()`
- Anomaly score = $|\text{predicted} - \text{observed}|$
- Training is significantly slower than classical models — plan for longer execution time

20 · hanr_ml + Conv1D – 1D Convolutional Network

A 1D Convolutional Neural Network (Conv1D) applies learned filters (kernels) along the time axis of a sequence. Each filter detects a specific local pattern — a shape, slope, or spike — regardless of its position in the series (translation invariance). Unlike LSTM, Conv1D does not maintain a hidden state across time steps; it captures local shape features through sliding filter operations. This makes it more efficient than LSTM for anomalies defined by local shape rather than long-range context. Used within `hanr_ml()`, Conv1D is trained to predict the next value from a window; large prediction residuals signal anomalies.

❏ **Key concept:** Conv1D outperformed LSTM on this dataset with the same training budget. Prefer Conv1D when anomalies have a distinct local shape (spikes, steps).

```
# Requer: library(daltoolboxdp)
model <- hanr_ml(
  ts_conv1d(ts_norm_gminmax(),
    input_size = 4,
    epochs = 10000))
model <- fit(model, dataset$serie)
```

```
## idx event  type
## 50 TRUE anomaly

## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100
```

Implementation

- Requires `daltoolboxdp` and `tspredict` packages
- `ts_norm_gminmax()` embeds normalization into the pipeline
- `input_size=4` defines the filter receptive field (window length)
- `epochs=10000` — same training budget as LSTM
- `fit()` trains the Conv1D filters via backpropagation
- `detect()` computes residuals and thresholds via `harutils()`
- Anomaly score = |predicted – observed|
- Parallelizable training — unlike RNNs, Conv1D does not require sequential computation

21 · hanc_ml + Majority – Baseline Classifier

`hanc_ml()` wraps any DALToolbox binary classifier and treats anomaly detection as a labeled classification problem. Unlike `hanr_ml()`, which uses prediction error as the anomaly score, `hanc_ml()` uses the positive class probability of the positive class (TRUE). It requires a labeled dataset (`examples_anomalies$tt`) split into train/test sets, with min-max normalization fitted on the training set and applied to both — preventing data leakage. The Majority classifier always predicts the most frequent class (FALSE) and serves as the absolute baseline: any useful model must outperform it.

 **Key concept:** The Majority classifier is an absolute baseline — it always predicts the most frequent class. Any useful model must outperform this result.

```
dataset <- examples_anomalies$tt
train <- dataset[1:80,]
test <- dataset[-(1:80),]
norm <- minmax()
norm <- fit(norm, train)
train_n <- transform(norm, train)
test_n <- transform(norm, test)
```

```
model <- hanc_ml(
  cla_majority("event",
    c("FALSE", "TRUE")))
model <- fit(model, train_n)
```

```
## Train: 0 TP, 0 FP
## (always predicts FALSE)
```

```
## Test confMatrix:
## detection TRUE FALSE
##   TRUE  0  0
##   FALSE 2 19
```

Step by step – Majority

- Classification uses `examples_anomalies$tt` (a labeled dataset)
- Split: 80 training / 21 test samples; min-max normalization applied
- `hanc_ml()` accepts any DALToolbox classifier
- Majority: always predicts the majority class (FALSE)
- Result: 0 TP, 0 FP — the minimum expected baseline

Regression vs. Classification

- `hanr_ml()`: uses prediction error as the anomaly score
- `hanc_ml()`: uses the positive class probability as the anomaly score

22 · hanc_ml + Decision Tree – Decision Tree Classifier

A Decision Tree recursively partitions the feature space using axis-aligned splits that maximize class separation (Gini impurity or information gain), producing human-readable if-then rules. Each leaf node assigns a class label or probability to observations that reach it. No normalization required — tree splits are scale-invariant. Struggles with class imbalance: anomalies are rare, so the tree rarely learns meaningful splits. Useful as an interpretable baseline before moving to ensemble methods.

❏ **Key concept:** The Decision Tree is interpretable and fast, but can struggle with imbalanced datasets that have few anomalies. It serves as a baseline before more sophisticated models such as RF and SVM.

```
train <- dataset[1:80,]  
test  <- dataset[-(1:80),]  
norm  <- minmax()  
norm  <- fit(norm, train)  
train_n <- transform(norm, train)  
test_n  <- transform(norm, test)
```

```
model <- hanc_ml(  
  cla_dtree("event",  
    c("FALSE", "TRUE"))  
)  
model <- fit(model, train_n)
```

```
## Train: 0 TP, 0 FP
```

```
## Test confMatrix:  
## detection TRUE FALSE  
##   TRUE  0  0  
##   FALSE  2 19
```

Step-by-step – Decision Tree

- `cla_dtree()`: decision tree classifier from DALToolbox
- Imbalanced dataset: very few anomalies versus many normal observations
- Failed on the test set (0 TP) — dataset is small and heavily imbalanced
- Useful as an interpretable baseline before moving to more complex models
- Compare with k-NN and RF to assess real performance gains

23 · hanc_ml + k-NN – k Nearest Neighbors

k-NN is a non-parametric, instance-based classifier — there is no explicit training phase; the model simply memorizes the training set. It makes predictions by finding the k most similar training examples (nearest neighbors) and assigning the majority class among them. Distance metric: typically Euclidean. Normalization is critical since k-NN is distance-sensitive. Works well when anomalies form distinct clusters in feature space.

❏ **Key concept:** k-NN classifies by similarity to labeled examples — it works well when anomalies form distinct clusters in feature space. Delivers excellent results on this dataset.

```
train <- dataset[1:80,]
test  <- dataset[-(1:80),]
norm  <- minmax()
norm  <- fit(norm, train)
train_n <- transform(norm, train)
test_n  <- transform(norm, test)

model <- hanc_ml(
  cla_knn("event",
    c("FALSE","TRUE"), k=3))
model <- fit(model, train_n)
```

```
## Train: 4 TP, 0 FP
## Test confMatrix:
## detection TRUE FALSE
##   TRUE  2  0
##   FALSE 0 19
```

Step-by-step – k-NN

- $k=3$: considers the 3 nearest neighbors
- Training: perfect classification (4 TP, 0 FP)
- Test: 2 TP, 0 FP — excellent generalization on this dataset
- Positive class probabilities above threshold → event detected
- Works well when anomalies form distinct clusters in feature space

24 · hanc_ml + Naive Bayes – Probabilistic Classifier

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the "naive" assumption that all features are conditionally independent given the class label.

$P(\text{class}|\text{features}) \propto P(\text{class}) \times \prod P(\text{feature}_i|\text{class})$. Gaussian NB assumes each feature follows a normal distribution per class. Despite its simplicity, it fails on time series data because lagged features are highly correlated — directly violating the independence assumption.

❏ **Key concept:** Naive Bayes assumes conditional independence between features — an assumption rarely valid in time series data. It failed on this dataset, confirming that temporal correlation plays a critical role in classification.

```
train <- dataset[1:80,]
test  <- dataset[-(1:80),]
norm  <- minmax()
norm  <- fit(norm, train)
train_n <- transform(norm, train)
test_n  <- transform(norm, test)
```

```
model <- hanc_ml(
  cla_nb("event",
    c("FALSE", "TRUE")))
model <- fit(model, train_n)
```

```
## Train: 0 TP, 0 FP
```

```
## Test confMatrix:
## detection TRUE FALSE
##   TRUE  0  0
##   FALSE 2 19
```

Step-by-step – Naive Bayes

- Assumes conditional independence between features
- Failed on test set (0 TP) — independence assumption is inadequate here
- Time series features are highly correlated with one another
- Useful as a simple probabilistic baseline
- Contrast with k-NN: similarity outperforms independence in this case

25 · hanc_ml + RF – Random Forest Classifier

Random Forest for classification builds an ensemble of decision trees, each trained on a bootstrap sample of the data. Each tree votes for a class label and the majority wins — the positive class vote proportion serves as the anomaly score. `mtry` controls the number of features considered at each split (randomness); `ntree` controls the number of trees (stability). Aggregating many trees reduces variance and handles class imbalance better than a single Decision Tree.

📌 **Key concept:** RF aggregates votes from diverse trees — each trained on a different bootstrap sample. The ensemble is more robust than any individual tree, especially on imbalanced datasets with few labeled anomalies.

```
model_rf <- hanc_ml(  
  cla_rf("event",  
    c("FALSE","TRUE"),  
    mtry=1, ntree=5))  
model_rf <- fit(model_rf, train_n)
```

```
## Train: 4 TP, 0 FP
```

```
## Test confMatrix:  
## detection TRUE FALSE  
##    TRUE  2   0  
##    FALSE  0  19
```

Step-by-step – RF Classification

- `mtry=1`: features per split
- `ntree=5`: number of trees
- Aggregating votes reduces variance
- Handles class imbalance better than a single tree
- Perfect train and test results: 4 TP / 2 TP, 0 FP

26 · hanc_ml + SVM – Support Vector Machine Classifier

SVM for classification finds the maximum-margin hyperplane that separates the two classes. The margin is the distance between the hyperplane and the nearest training points (support vectors). A soft-margin SVM (controlled by cost C) allows some misclassifications to handle non-separable data. The kernel trick can extend SVM to non-linear boundaries (RBF or linear kernel). cost controls the penalty for misclassification — high C = tighter fit to training data. epsilon=0.0 sets pure classification mode (no regression tube). Works well with small datasets and few labeled anomalies.

❏ **Key concept:** SVM finds the maximum-margin hyperplane — the decision boundary that is as far as possible from both classes. This margin maximization makes it robust to small datasets and class imbalance. Ref: Cortes & Vapnik, 1995.

```
model_svm <- hanc_ml(  
  cla_svm("event",  
    c("FALSE","TRUE"),  
    epsilon=0.0, cost=20))  
model_svm <- fit(model_svm, train_n)
```

```
## Train: 4 TP, 0 FP
```

```
## Test confMatrix:  
## detection TRUE FALSE  
##   TRUE  2  0  
##   FALSE 0 19
```

Step-by-step – SVM

- cost=20 high penalty for misclassification
- epsilon=0.0 pure classification mode
- maximum-margin principle robust to outliers near decision boundary
- works well with small datasets and few labeled anomalies
- same result as RF: 4 TP train, 2 TP test, 0 FP

27 · hanc_ml + MLP – Neural Network Classifier

The same MLP architecture used for regression can be adapted for binary classification by replacing the linear output with a softmax activation and using cross-entropy loss instead of MSE. The output is a probability distribution over classes — the positive class probability (TRUE) serves as the anomaly score. L2 regularization (decay) prevents overfitting.

❏ **Key concept:** MLP as a classifier failed on this small dataset. Neural networks generally require more labeled examples to generalize effectively — RF and SVM are preferable when labeled data is scarce.

```
train <- dataset[1:80,]
test  <- dataset[-(1:80),]
norm  <- minmax()
norm  <- fit(norm, train)
train_n <- transform(norm, train)
test_n  <- transform(norm, test)

model <- hanc_ml(
  cla_mlp("event",
    c("FALSE", "TRUE"),
    size=3, decay=0.03))
model <- fit(model, train_n)
```

```
## Train: 0 TP, 0 FP
```

```
## Test confMatrix:
```

```
## detection TRUE FALSE
```

```
##   TRUE  0  0
```

```
##   FALSE  2 19
```

```
## (failed — insufficient data)
```

Step-by-step – MLP Classification

- size=3: hidden neurons; decay=0.03: L2 regularization
- Failed on the test set (0 TP) — likely needs more training data
- Small datasets are disadvantageous for neural networks
- Contrast with RF and SVM: simpler models generalized significantly better

28 · hanct_kmeans() – Anomalies via k-means

`hanct_kmeans()` applies k-means clustering to individual observations (`seq=1`) or sliding-window subsequences (`seq>1`). Observations that lie far from their nearest centroid are flagged as anomalies. The implementation wraps `stats::kmeans` and uses `harutils()` to summarize distances and set thresholds. With `seq=1` it detects point anomalies; with `seq>1` it detects discords (rare subsequences).

```
# k-means: point anomalies (seq=1)
dataset <- examples_anomalies$simple
```

```
model <- hanct_kmeans(1)
set_example_seed()
model <- fit(model, dataset$serie)
```

```
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))
## idx event  type
## 1  50  TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100
```

```
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Step-by-step – k-means (seq=1)

- Uses `examples_anomalies$simple` (point anomalies)
- `hanct_kmeans(1)` clusters individual observations
- `fit()` runs k-means on the series
- `detect()` computes distance to nearest centroid and flags outliers via `harutils()`
- Perfect result: 1 TP, 0 FP

❏ **Key concept:** `seq=1` → point anomaly detection; `seq>1` → discord (subsequence) detection. Only the `seq` argument changes between the two modes.

29 · hanct_kmeans() – Discords via k-means (seq>1)

`hanct_kmeans()` with `seq>1` detects discords — rare subsequences — by clustering sliding windows of length `seq`. Windows that lie far from their nearest centroid are flagged as discords. Uses `examples_anomalies$sequence` (a series with a repeating pattern and one anomalous subsequence). The implementation wraps `stats::kmeans` and thresholds via `harutils()`.

❏ **Key concept:** Discord detection is a different task from anomaly detection — the goal is to find the most unusual subsequence, not a point outlier. The `confMatrix` result reflects a temporal offset, not a failure.

```
# k-means: discords (seq=3)
dataset <-
examples_anomalies$sequence

model <- hanct_kmeans(3)
set_example_seed()
model <- fit(model, dataset$serie)

detection <- detect(model,
dataset$serie)
print(detection | >
dplyr::filter(event == TRUE))
## idx event  type seq seqlen
## 1 51 TRUE discord 3 3
```

Step-by-step – k-means Discords

- Uses `examples_anomalies$sequence` (repeating pattern with one discord)
- `hanct_kmeans(3)` clusters windows of length 3
- `fit()` runs k-means on subsequences
- `detect()` flags windows far from any centroid
- Result: 0 TP, 1 FP — the discord is detected but offset from the ground-truth label

30 · hanct_dtw() – Anomalies via DTW

DTW Theory

DTW (Dynamic Time Warping) is a distance measure that allows **flexible temporal alignment** between two sequences — unlike Euclidean distance which requires strict point-to-point correspondence. It finds the optimal **"warping path"** between two series by minimizing total distance, effectively stretching or compressing one sequence in time to match the other.

In anomaly detection, DTW-based clustering assigns observations or subsequences to clusters; those with **large DTW distance to their nearest centroid** are flagged as anomalies. DTW excels when series exhibit **phase shifts, speed variations, or local distortions**. Implemented via `dtwclust`; higher computational cost than Euclidean distance.

Ref: Berndt & Clifford, KDD 1994; Sakoe & Chiba, 1978.

Key concept: DTW finds the optimal warping path — more robust than Euclidean distance for series with local distortions.

seq=1 → point anomalies; **seq>1** → discord detection.

Code & Output

```
# DTW: point anomalies (seq=1)
dataset <- examples_anomalies$simple

model <- hanct_dtw(1)
set_example_seed()
model <- fit(model, dataset$serie)
detection <- detect(model, dataset$serie)
evaluation <- evaluate(model,
  detection$event, dataset$event)
```

```
## idx event  type
## 50 TRUE anomaly

## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100
```

Step-by-step – DTW Anomalies

- Uses `examples_anomalies$simple`
- `hanct_dtw(1)` clusters individual observations using DTW distance
- `set_example_seed()` ensures reproducibility
- `fit()` runs DTW-based clustering
- `detect()` flags observations far from nearest centroid via `harutils()`
- Result: **1 TP, 0 FP**

31 · hanct_dtw() – Discords via DTW (seq=3)

`hanct_dtw()` with `seq>1` detects discords by applying DTW clustering over sliding-window subsequences. Windows with large DTW distance to their nearest centroid are flagged as discords. Uses `examples_anomalies$sequence`. Implementation wraps `dtwclust` and thresholds via `harutils()`. DTW is preferred over k-means for discord discovery when the shape of subsequences matters more than their magnitude.

❏ **Key concept:** DTW discord discovery is shape-sensitive — it finds the subsequence whose shape is most unlike all others, regardless of temporal position. The offset in the `confMatrix` is expected for subsequence-level detection.

```
# DTW: discords (seq=3)
dataset <-
examples_anomalies$sequence

model <- hanct_dtw(3)
set_example_seed()
model <- fit(model, dataset$serie)

detection <- detect(model,
dataset$serie)
print(detection | >
dplyr::filter(event == TRUE))
## idx event   type seq seqlen
## 1  51  TRUE discord  3     3
```

Step-by-step – DTW Discords

- Uses `examples_anomalies$sequence` as the target dataset
- `hanct_dtw(3)` clusters windows of length 3 using DTW distance
- `fit()` runs DTW-based clustering on subsequences
- `detect()` flags windows far from any centroid
- Result: 0 TP, 1 FP — temporal offset between detected and labeled discord

32 · har_ensemble() – Majority Voting

`har_ensemble()` combines multiple base detectors via majority voting — a point is flagged only if the majority of detectors agree. Individual detectors have distinct strengths and weaknesses; combining detectors from different families (regression-based, spectral, statistical) reduces false positives and improves robustness. The effectiveness of an ensemble relies on detector diversity — each family brings a unique perspective, ensuring comprehensive coverage of anomaly types. This also reduces variance without increasing bias (bias-variance tradeoff), leading to more stable and accurate anomaly detection.

❏ **Key concept:** Majority voting requires consensus — a point is flagged only if the majority of detectors agree, reducing false positives from any single method.

```
model <- har_ensemble(  
  hanr_fbiad(),  
  hanr_arima(),  
  hanr_emd()  
)  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
## 50  TRUE anomaly  
  
## detection TRUE FALSE  
##   TRUE  1   0  
##  FALSE  0 100
```

Step by Step

- `har_ensemble()` accepts any number of base detectors
- Each base detector casts one vote
- The final event reflects majority consensus
- Base detectors: FBIAD, ARIMA, and EMD — three distinct method families
- Perfect result: 1 TP, 0 FP

33 · har_ensemble() Fuzzy – Temporal Tolerance

In ensemble anomaly detection, individual base detectors often identify the same underlying anomaly but with slight temporal shifts. Temporal fuzzification merges detections that occur within a defined `time_tolerance` window before the voting rule is applied — transforming exact timestamps into temporal neighborhoods. This prevents double-counting a single anomaly flagged at slightly different moments by different detectors, and ensures detected anomalies align with the granularity of ground-truth labels. Particularly useful when detectors like ARIMA consistently identify events a few steps after their actual occurrence.

❏ **Key concept:** Temporal fuzzification merges nearby detections before voting — prevents double counting when different detectors flag the same event at slightly different moments.

```
model <- har_ensemble(  
  hanr_fbiad(),  
  hanr_arima(),  
  hanr_emd())  
  
# Merge detections within 10 steps  
model$time_tolerance <- 10  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
## 50  TRUE anomaly  
  
## detection TRUE FALSE  
##   TRUE  1   0  
##  FALSE  0 100
```

Step by step

- Same setup as the basic ensemble
- Add `model$time_tolerance <- 10` after creating the ensemble
- Detections within 10 time steps are merged before voting
- Reduces duplicate triggers from detectors that disagree slightly in time
- Result: 1 TP, 0 FP (same as simple ensemble here)

41 · hmu_pca() – Multivariate Detection via PCA

Most anomaly detection methods operate on a single time series. When multiple correlated variables are observed simultaneously, anomalies may only be visible in the joint behavior — not in any individual series.

PCA learns a low-dimensional subspace that captures the normal variance of the data. Each new observation is projected onto this subspace and reconstructed; the reconstruction error (L2 norm of residual) measures how far the observation deviates from the normal subspace. Observations with high reconstruction error are flagged as anomalies.

❏ **Key concept:** PCA flags observations far from the learned normal subspace. Anomalies in multivariate data may only be visible in the joint behavior of variables — not in any single series alone.

Ref: Jolliffe, I.T. (2002). *Principal Component Analysis*. Springer, 2nd ed. | Shyu et al. (2003). *A Novel Anomaly Detection Scheme Based on Principal Component Classifier*. ICDM.

```
data("examples_harbinger")
dataset <-
examples_harbinger$multidimensional
dataset$event <- FALSE
dataset$event[c(101, 128, 167)] <- TRUE

model <- fit(hmu_pca(),
             dataset[, 1:2])
```

```
## Events at idx: 101, 128, 167
```

```
## Reconstruction error plotted
## per dimension via har_plot()
## with ylab("serie") / ylab("x")
```

Step-by-step

- Uses `examples_harbinger$multidimensional` (2 variables)
- Event labels created manually for the demo: events at idx=101, 128, 167
- `hmu_pca()` is the Harbinger function for multivariate anomaly detection.
- `hmu_pca()` projects observations onto principal components
- `fit()` and `detect()` receive a dataframe with multiple columns
- PCA reconstruction error serves as the anomaly score
- Plot detections for each dimension separately using `ylab()`

34 · Basic Autoencoder – Window Reconstruction

An autoencoder is a neural network trained to compress an input into a compact latent representation (encoding) and then reconstruct it (decoding). When trained exclusively on normal data, the network learns to reconstruct normal patterns well — but fails on anomalous inputs, producing high reconstruction error.

`han_autoencoder()` is the unified interface for all variants in Harbinger — only the `type` argument changes:

- `autoenc_ed` — basic dense encoder-decoder
- `autoenc_denoise_ed` — denoising (trained on corrupted inputs)
- `autoenc_conv_ed` — convolutional (Conv1D layers)
- `autoenc_lstm_ed` — LSTM (long-range temporal dependencies)
- `autoenc_stacked_ed` — stacked (multiple hidden layers)
- `autoenc_variational_ed` — VAE (probabilistic latent space)
- `autoenc_adv_ed` — adversarial (AAE regularization)

 **Key concept:** All autoencoder variants share the same `han_autoencoder()` interface — only the `type` argument changes. The encoder compresses the window into a latent code; the decoder reconstructs it. High reconstruction error = anomaly.

Requires `daltoolboxdp`. Ref: Sakurada & Yairi (2014).

```
# Requer: library(daltoolboxdp)
# type argument selects the variant
model <- han_autoencoder(
  3, 2, autoenc_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)
```

```
## idx event  type
##  51  TRUE anomaly
## (1-step offset from idx=50)

## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  1  99
```

Step by step – Basic Autoencoder

- Requires `daltoolboxdp` in addition to `daltoolbox`
- Arguments: `encoder_size=3`, `decoder_size=2`, `type`, `num_epochs=1500`
- Reconstructs sliding windows — high reconstruction error signals an anomaly
- Detected `idx=51` (close to ground truth at 50): 0 TP, 1 FP
- A one-step offset is common with basic autoencoders

35 · Denoising Autoencoder – Noise Robustness

A Denoising Autoencoder (DAE) is trained to reconstruct clean inputs from artificially corrupted versions. By learning to ignore noise during training, the model develops a more robust internal representation of normal patterns. At inference time, the original (uncorrupted) window is fed — high reconstruction error signals an anomaly. Key difference from basic AE: input = corrupted window → reconstruct clean window. The advantage over the basic AE emerges on real-world noisy series with sensor drift or measurement jitter.

❏ **Key concept:** A denoising autoencoder adds robustness by training on corrupted inputs — it learns to ignore ordinary noise and focus on the underlying structure. This provides a real advantage on series affected by measurement noise.

```
model <- han_autoencoder(  
  3, 2, autoenc_denoise_ed,  
  num_epochs = 1500)  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
##  51  TRUE anomaly  
  
## detection TRUE FALSE  
##    TRUE  0   1  
##   FALSE  1  99
```

Step by step – Denoising AE

- Learns to reconstruct clean inputs from noisy windows
- Trained with artificially corrupted inputs
- More robust to noise than the basic autoencoder
- Same result on this dataset: idx=51, 0 TP, 1 FP
- The difference emerges on series with real-world noise

36 · Convolutional Autoencoder – Local Patterns

A Convolutional Autoencoder (Conv AE) replaces the dense layers of a standard autoencoder with 1D convolutional filters (Conv1D). Filters slide over the time window detecting local features (peaks, drops, shape changes) with translation invariance — the same feature is detected regardless of its exact position within the window. Encoder: Conv1D layers progressively compress the window into a latent representation. Decoder: transposed Conv1D layers reconstruct the window from the latent code. Anomaly score: reconstruction error. Best suited for point anomalies and shape anomalies with a localized signature.

 **Key concept:** The convolutional autoencoder uses Conv1D filters to capture local patterns within the time window. Translation invariance makes it robust to small temporal shifts in the position of an anomaly.

```
model <- han_autoencoder(  
  3, 2, autoenc_conv_ed,  
  num_epochs = 1500)  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
##  50  TRUE anomaly  
  
## detection TRUE FALSE  
##   TRUE   1    0  
##  FALSE   0  100
```

Step-by-step – Conv Autoencoder

- Uses Conv1D layers in both the encoder and decoder
- Captures local patterns and provides translation invariance
- Perfect result: 1 TP, 0 FP (detected exactly idx=50)
- Outperforms the basic autoencoder on this dataset
- Best suited for anomalies with a localized shape

37 · LSTM Autoencoder – Temporal Dependencies

An LSTM Autoencoder replaces the dense or convolutional layers with Long Short-Term Memory (LSTM) units. The encoder LSTM reads the input window step by step, compressing the entire sequence into a fixed-size context vector (latent state). The decoder LSTM reconstructs the sequence from the context vector, step by step. The latent representation encodes the temporal order and long-range context of the entire window — not just local features. Best suited for contextual anomalies where the anomaly is defined by its relationship to surrounding values over a long horizon. May exhibit a one-step temporal offset in the detected anomaly index.

```
model <- han_autoencoder(  
  3, 2, autoenc_lstm_ed,  
  num_epochs = 1500)  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
##  51  TRUE anomaly  
## (1-step offset from idx=50)  
  
## detection TRUE FALSE  
##   TRUE  0   1  
##  FALSE  1  99
```

Step-by-step – LSTM Autoencoder

- LSTM learns temporal dependencies by encoding and decoding full sequences
- Optimizes reconstruction of complete time sequences
- Detected idx=51 (0 TP, 1 FP) — one-step offset from the true anomaly
- Contrast with Conv AE: LSTM was less precise on this dataset
- Best suited for long-range contextual anomalies

 **Key concept:** The LSTM autoencoder captures long-range temporal dependencies in the latent representation. It excels at contextual anomalies — but may exhibit a temporal shift in the detection index.

38 · Stacked Autoencoder – Deep Representation

A Stacked Autoencoder extends the basic autoencoder by adding multiple hidden layers in both the encoder and decoder. Each additional layer allows the network to learn increasingly abstract representations of the input data.

- Encoder: input → hidden layer 1 → hidden layer 2 → ... → latent code (bottleneck)
- Decoder: latent code → hidden layer n → ... → reconstructed output
- Deeper architectures capture complex non-linear structure that simple models cannot represent
- Trade-off: higher sensitivity comes at the cost of more false positives — threshold tuning is essential
- Result: 1 TP, 2 FP (idx=26, 50, 77 detected)

 **Key concept:** Stacked AE deepens the latent representation through multiple layers, capturing richer non-linear structure. This makes it more sensitive to subtle anomalies, but threshold tuning is essential to keep false positives in check.

```
model <- han_autoencoder(  
  3, 2, autoenc_stacked_ed,  
  num_epochs = 1500)  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
## 26 TRUE anomaly  
## 50 TRUE anomaly  
## 77 TRUE anomaly  
  
## detection TRUE FALSE  
##   TRUE  1   2  
##  FALSE  0  98
```

Step-by-step – Stacked AE

- Deepens encoder/decoder layers to capture richer non-linear structure
- Detected 3 events (idx=26, 50, 77): 1 TP, 2 FP
- Deeper architecture → higher sensitivity, but also more false positives
- Threshold tuning required to reduce FP rate
- Well-suited for detecting subtle anomalies

39 · Variational Autoencoder (VAE) – Probabilistic Latent Space

A Variational Autoencoder (VAE) is a generative model that extends the standard autoencoder by learning a probabilistic latent space instead of a fixed point encoding. The encoder outputs a distribution parameterized by mean (μ) and variance (σ^2). A latent vector z is sampled from $N(\mu, \sigma^2)$ using the reparameterization trick. Loss = reconstruction error + KL divergence (regularizes the latent distribution toward $N(0,1)$). Anomalous windows produce both high reconstruction error AND high uncertainty (large σ^2) — two complementary signals. Key advantage over deterministic AEs: the VAE quantifies uncertainty in its detections.

❏ **Key concept:** The VAE learns a probabilistic latent distribution (μ, σ^2). Anomalies exhibit high uncertainty and elevated reconstruction error. Ref: Kingma & Welling, 2013.

```
model <- han_autoencoder(  
  3, 2, autoenc_variational_ed,  
  num_epochs = 1500)  
model <- fit(model, dataset$serie)
```

```
## idx event  type  
## 50 TRUE anomaly  
## 58 TRUE anomaly  
## 85 TRUE anomaly  
  
## detection TRUE FALSE  
##   TRUE  1   2  
##  FALSE 0  98
```

Step-by-step – VAE

- Learns a probabilistic latent space (mean + variance)
- Reconstruction error and uncertainty signal anomalies
- Detected 3 events (idx=50, 58, 85): 1 TP, 2 FP
- Similar sensitivity to Stacked AE
- Advantage: quantifies uncertainty in detection

40 · Adversarial Autoencoder (AAE)

An Adversarial Autoencoder (AAE) combines an autoencoder with a GAN-style discriminator applied to the latent space. Instead of KL divergence (as in VAE), adversarial training forces the encoder's output to match a chosen prior distribution (e.g., Gaussian).

Three components:

1. **Encoder:** maps input windows to latent vectors z
2. **Decoder:** reconstructs the window from z
3. **Discriminator:** distinguishes encoder latent vectors from prior samples — the encoder is trained to fool the discriminator

More flexible than VAE: can match any prior distribution. Training instability is a known limitation.

📌 **Key concept:** AAE regularizes the latent space through adversarial training, forcing the latent distribution to follow a prior. Windows that deviate from the learned manifold yield elevated reconstruction error. Ref: Makhzani et al., arXiv 2016.

```
# Encoder + Decoder + Discriminator
model <- han_autoencoder(
  3, 2, autoenc_adv_ed,
  num_epochs = 1500)
model <- fit(model, dataset$serie)
```

```
## idx event  type
## 19 TRUE anomaly
## 44 TRUE anomaly

## detection TRUE FALSE
##   TRUE  0   2
##  FALSE  1  98
```

Step-by-step

- AAE augments the encoder-decoder with an adversarial objective applied to the latent code
- A discriminator distinguishes latent samples produced by the encoder from samples drawn from a prior distribution (e.g., Gaussian)
- The encoder is trained to fool the discriminator — aligning the posterior with the prior
- Normal patterns remain on a compact manifold; anomalies exhibit higher reconstruction error
- Detected idx=19 and 44 (0 TP, 2 FP on this dataset)

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

